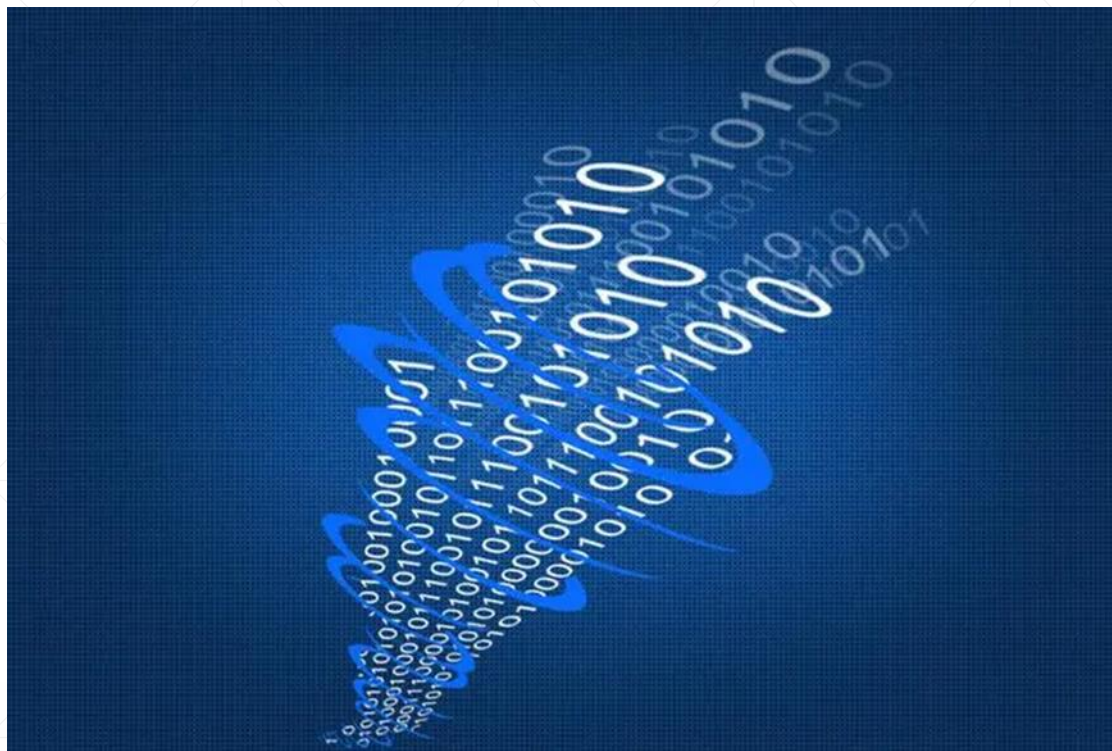




使用流存取文件

北京理工大学计算机学院
金旭亮

读写文件



要从文件中以字节为单位读写二进制数据，可以使用 `FileInputStream` 和 `FileOutputStream`。



将文件路径字符串传给 `FileInputStream` 的构造函数，创建一个它的实例，之后，就可以调用它的 `read()` 系列方法读取数据。



`FileOutputStream` 的使用方法是类似的。

```
//展示文件输入流的基本用法
public class FileInputStreamTest {
    //流的许多方法, 都会抛出IOException异常, 所以要加以声明
    public static void main(String[] args) throws IOException {
        //创建字节输入流
        var fis = new FileInputStream( name: "./src/fileStream/FileInputStreamTest.java");
        //创建一个长度为1024的数组
        byte[] bbuf = new byte[1024];
        //用于保存实际读取的字节数
        int hasRead = 0;
        //使用循环来重复读取数据
        while ((hasRead = fis.read(bbuf)) > 0) {
            //将字节数组转换成字符串输入
            System.out.print(new String(bbuf, offset: 0, hasRead));
        }
        fis.close();
    }
}
```

虽然可以使用循环来读取文件, 但本例所展示的方式是比较低效的, 因为它没有充分用好缓存, 并且上述代码执行时会阻塞调用者线程。

通过字节流实现文件复制

```
//文件输入流与文件输出流配合使用，实现文件复制
public class FileOutputStreamTest {
    public static void main(String[] args) throws IOException {
        FileInputStream fis = null;
        FileOutputStream fos = null;
        try {
            //创建字节输入流
            fis = new FileInputStream( name: "./src/fileStream/FileOutputStreamTest.java");
            //创建字节输出流
            fos = new FileOutputStream( name: "newFile.txt");
            byte[] bbuf = new byte[32];
            int hasRead = 0;
            //循环从输入流中取出数据
            while ((hasRead = fis.read(bbuf)) > 0) {
                //每读取一次，即写入文件输出流，读了多少，就写多少。
                fos.write(bbuf, off: 0, hasRead);
            }
        }
```

FileOutputStream.java

必须关闭流对象

```
        } catch (IOException ioe) {
            ioe.printStackTrace();
        } finally {
            //使用finally块来关闭文件输入流
            if (fis != null) {
                fis.close();
            }
            //使用finally块来关闭文件输出流
            if (fos != null) {
                fos.close();
            }
        }
    }
}
```

直接使用文件流存在的问题



使用FileInputStream和FileOutputStream存取文件，是以字节为基本单位的，固然非常灵活与方便，但有一个比较大的缺陷，就是效率比较低。



为了提升效率，通常会使用缓存，JDK提供了两种数据缓存组件：BufferedInputStream和BufferedOutputStream，可用于对文件输入流和输出流进行封装。

启用了缓存功能的文件流

文件输入流与缓存输入流需要“串联”（或者说“嵌套组合”）在一起，文件输入流负责读入数据，缓冲输入流则“暂存”这些数据，“积攒”多了之后，一次性地提交给使用者。



```
var in = new BufferedInputStream(new FileInputStream("文件名"));
```



默认情况下，数据缓存区的大小为8192字节。

使用缓存流复制文件



请看示例CopyFileDemo.java的运行截图：

```
Run: CopyFileDemo x
"C:\Program Files\Java\jdk-17\bin\java.exe"

没有使用缓存复制文件 ...
要复制的文件大小为: 465126 字节。
耗费时间 1265.2539 毫秒。

使用缓存复制文件 ...
耗费时间: 1.6097 毫秒。

Process finished with exit code 0
```



复制文件时，使用不使用缓存，性能相当极大！

最慢的复制文件方式

```
public static void fileCopyNoBuffer() {  
    System.out.println("\n没有使用缓存复制文件 ...");  
    long startTime, elapsedTime;  
    File fileIn = new File(inFileStr);  
    System.out.println("要复制的文件大小为: " + fileIn.length() + " 字节。");  
    //创建输入流与输出流  
    try (FileInputStream in = new FileInputStream(inFileStr);  
        FileOutputStream out = new FileOutputStream(outFileStr)) {  
        startTime = System.nanoTime();  
        int byteRead;  
        //从输入流中一个字节一个字节地读 (这是最慢的, 效率最低的一种方式)  
        while ((byteRead = in.read()) != -1) {  
            //写入到输出流中  
            out.write(byteRead);  
        }  
        elapsedTime = System.nanoTime() - startTime;  
        System.out.println("耗费时间 " + (elapsedTime / 1000000.0) + " 毫秒。");  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```



相比访问内存，磁盘IO操作是相当慢地，这里一个字节一个字节地读写，会带来大量的不必要的IO操作，所以运行速度很慢。

利用“数据缓存”加速文件复制



```
public static void fileCopyWithBufferAndArray() {
    System.out.println("\n使用缓存复制文件 ...");
    long startTime, elapsedTime;
    startTime = System.nanoTime();
    //基于文件输入和输出流，构建缓冲流，并且自动关闭
    try (var in = new BufferedInputStream(new FileInputStream(inFileStr));
        var out = new BufferedOutputStream(new FileOutputStream(outFileStr))) {
        //一次最多可读取4000字节
        byte[] byteBuf = new byte[4000];
        int numBytesRead;
        while ((numBytesRead = in.read(byteBuf)) != -1) {
            out.write(byteBuf, off: 0, numBytesRead);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
    elapsedTime = System.nanoTime() - startTime;
    System.out.println("耗费时间: " + (elapsedTime / 1000000.0) + " 毫秒。");
}
```

示例中一次可以读取4000字节，大大减少了IO操作，所以性能提升明显。



复制文件的简便方法

JDK 9+提供了一个`transferTo()`方法，可以方便地在输入流和输出流之间传送数据，它内置了数据缓存功能，使用它复制文件代码较少。

```
private static void fileCopyUseStreamTransferTo() {  
    System.out.println("\n使用transferTo方法复制文件 ...");  
    long startTime, elapsedTime;  
    startTime = System.nanoTime();  
    try (var inputStream = new FileInputStream(inFileStr);  
         var outputStream = new FileOutputStream(outFileStr)) {  
        var result :long = inputStream.transferTo(outputStream);  
        System.out.println("复制字节数: " + result);  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
    elapsedTime = System.nanoTime() - startTime;  
    System.out.println("耗时时间: " + (elapsedTime / 1000000.0) + " 毫秒。");  
}
```

可以看到，即使使用原始的输入输出流复制文件，性能也还可以。

```
Run: CopyFileDemo x  
"C:\Program Files\Java\jdk-17\bin\  
使用transferTo方法复制文件 ...  
复制字节数: 465126  
耗时时间: 7.899 毫秒。  
Process finished with exit code 0
```